CSE 4802: Software Maintenance Lab 3

Transformation, Logging, Tracing, and Profiling Tools

Project: Inventory Maintenance Lab | Student: _____ | ID: _____ | Date: 20 August 2026

1. Objective

This lab applies transformation and execution-observation tools to a working Python inventory and order-management project. The tools make runtime behavior visible through structured logs, line-level traces, execution timelines, and profile visualizations.

2. Selected project

The selected project is a SQLite-backed inventory CLI supporting product creation, restocking, order creation with stock validation, low-stock reporting, and summary reporting. lab/target.py creates three products and processes three orders so every tool observes the same workload.

Area	Implementation
Persistence	SQLite database with constraints
Business logic	inventory/service.py service functions
Interface	python -m inventory CLI
Verification	Three standard-library unit tests

3. Baseline execution

```
python lab\target.py
{'products': 3, 'units': 65, 'orders': 3, 'revenue_cents': 25343, 'low_stock_items': 0}

python -m unittest discover -s tests -v
Ran 3 tests in 0.002s
OK
```

The workload completes successfully. Revenue is 25,343 cents ($253.43), and no product is at or below its reorder threshold after processing.

4. Loguru

Loguru provides concise application-level event logging. The experiment records the start and successful completion of the workload in lab/loguru_output.log, with diagnose=False to avoid exposing sensitive variables.

```
python -m pip install -r requirements-lab.txt
python lab/loguru_demo.py
```

Maintenance insight: logs provide a durable event history that can be correlated with a customer report or production incident. They show whether the workload started, completed, or failed before a particular operation.

5. PySnooper

PySnooper traces source lines and variable changes. It is applied only to traced_process_orders so the output remains focused on a suspicious section.

```
python lab/pysnooper_demo.py
Get-Content lab/pysnooper_output.log
```

Maintenance insight: the trace exposes the order of calls, stock and order quantities, and returned values. It can identify the exact line that changed an incorrect intermediate value.

6. VizTracer

VizTracer records a timeline and call hierarchy without requiring source changes to the target workload.

```
viztracer -o lab/viztracer_output.json lab/target.py
```

Maintenance insight: the timeline shows whether time is spent in database setup, stock validation, order insertion, or report generation, and makes nested call relationships visible.

7. cProfile and SnakeViz

SnakeViz visualizes a standard cProfile output file. ncalls is the call count, tottime is time inside a function, and cumtime includes callees.

```
python -m cProfile -o lab/inventory.profile lab/target.py
snakeviz lab/inventory.profile
```

Maintenance insight: larger icicle blocks or sunburst sectors represent more cumulative execution time, helping a maintainer choose optimization targets based on evidence.

8. Alternatives and preference

Tool	Alternatives	Preferred use
Loguru	logging, structlog	Concise structured application logs
PySnooper	pdb, breakpoint(), IDE debugger	Focused reproducible traces
VizTracer	Pyinstrument, Scalene	Execution order and call timelines
SnakeViz	Tuna, pstats	Visual exploration of cProfile

For this project, I would keep Loguru for operational logging, use an IDE debugger for interactive debugging, and use cProfile/SnakeViz for routine performance maintenance. VizTracer is

preferable when call order or nested execution is central.

9. Environment note

The baseline workload and unit tests ran successfully. The analysis environment could not reach PyPI, so the four optional packages could not be installed here. The repository includes requirements-lab.txt, lab/README.md, and runnable experiment scripts. Run the commands below on a network-enabled machine to generate the tool-specific artifacts.

```
python -m pip install -r requirements-lab.txt
python lab/loguru_demo.py
python lab/pysnooper_demo.py
viztracer -o lab/viztracer_output.json lab/target.py
python -m cProfile -o lab/inventory.profile lab/target.py
snakeviz lab/inventory.profile
```

10. Conclusion

Logs answer what happened, line tracing answers how values changed, timeline tracing answers when and in what order calls ran, and profiling answers where runtime was spent. Together they provide complementary evidence for diagnosing defects and prioritizing maintenance work.